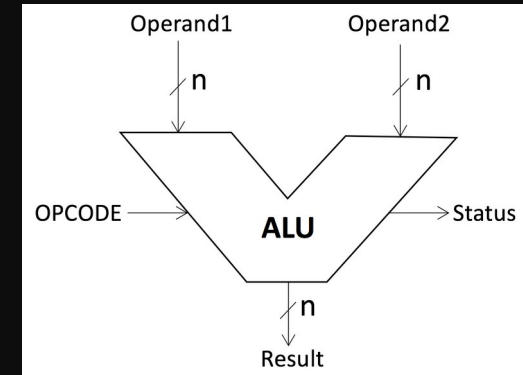


Lecture 10

Topics: Arithmetic microoperations, Logic microoperations



Probir Roy
Assistant professor,
CIS

CIS-310: Computer Organization & Assembly Language

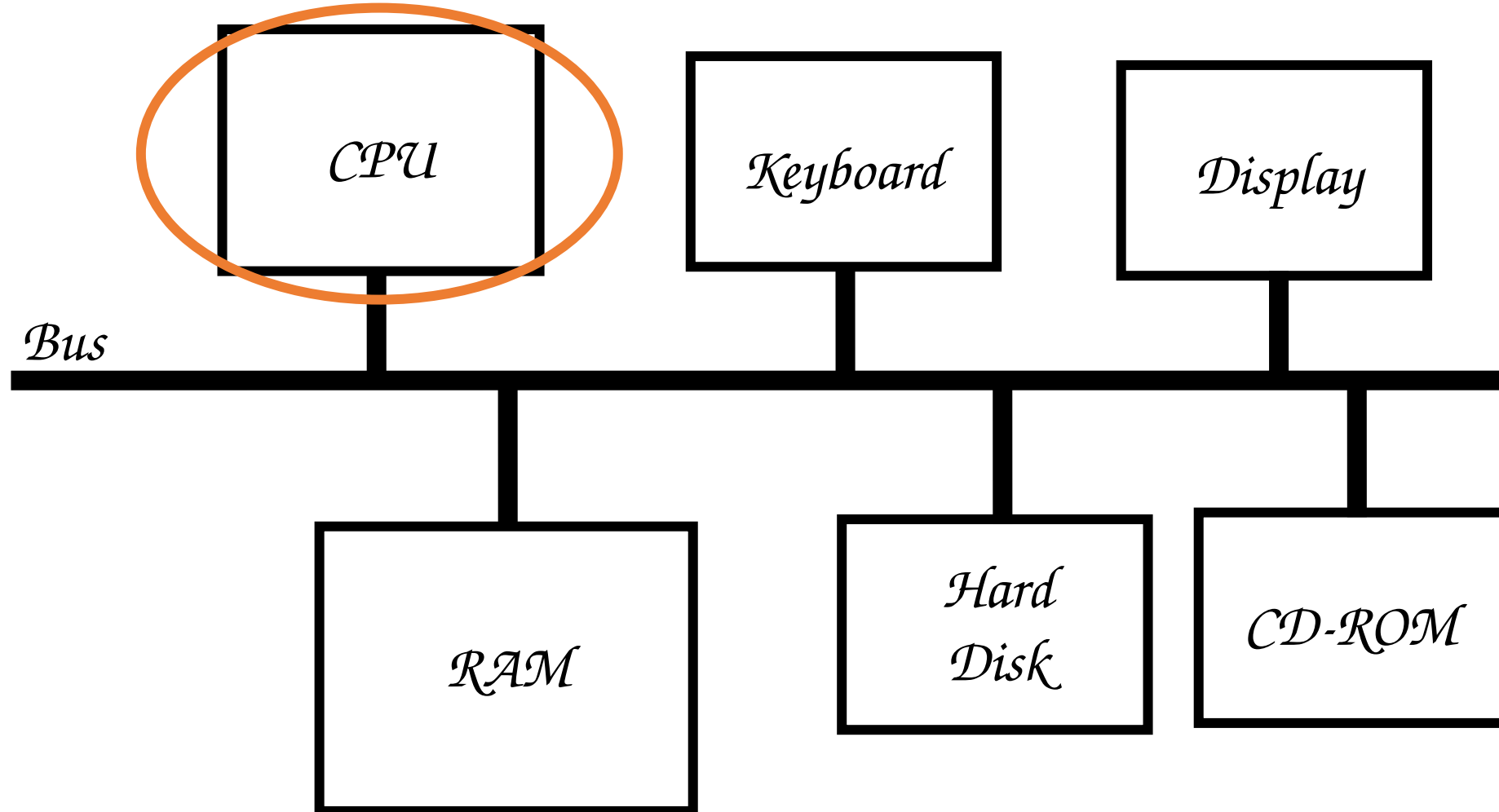
MW



Reading list

- **M. Mano, Computer System Architecture, 3rd ed, 1993.**

Computer Architecture



Inside the CPU

- The CPU is the brain of the computer.
- It is the part that actually executes the instructions.
- Let's take a look inside.

Memory Registers

Register 0

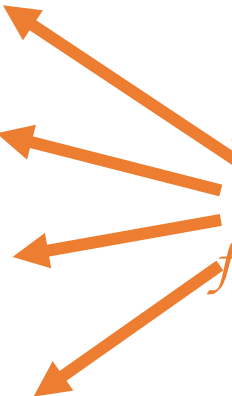
Register 1

Register 2

Register 3

Temporary Memory.

*Computer “Loads” data from
RAM to registers (via cache),
performs operations on data in
registers, and “stores” results
from registers back to RAM (via
cache)*



Memory Registers

Register 0

Register 1

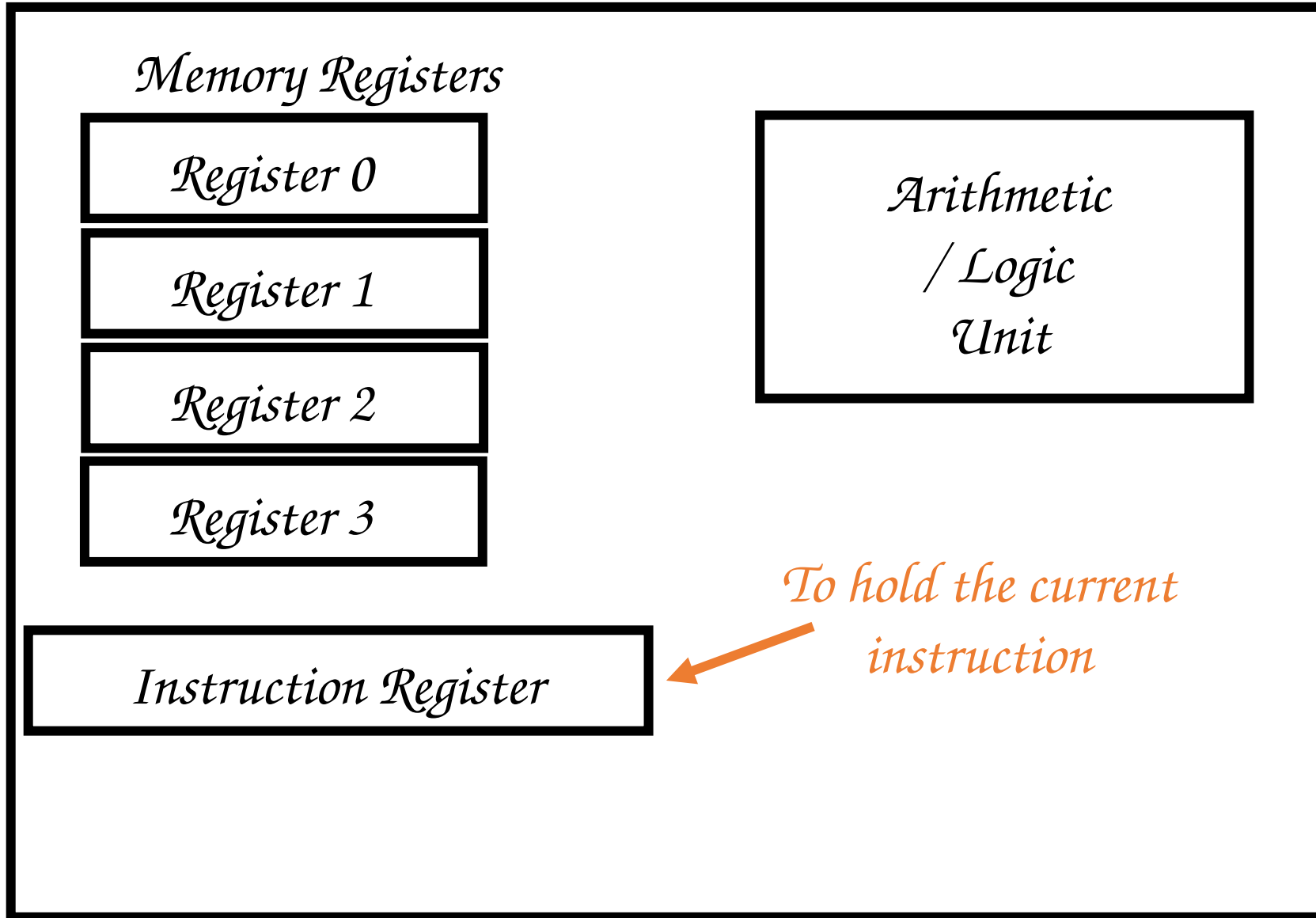
Register 2

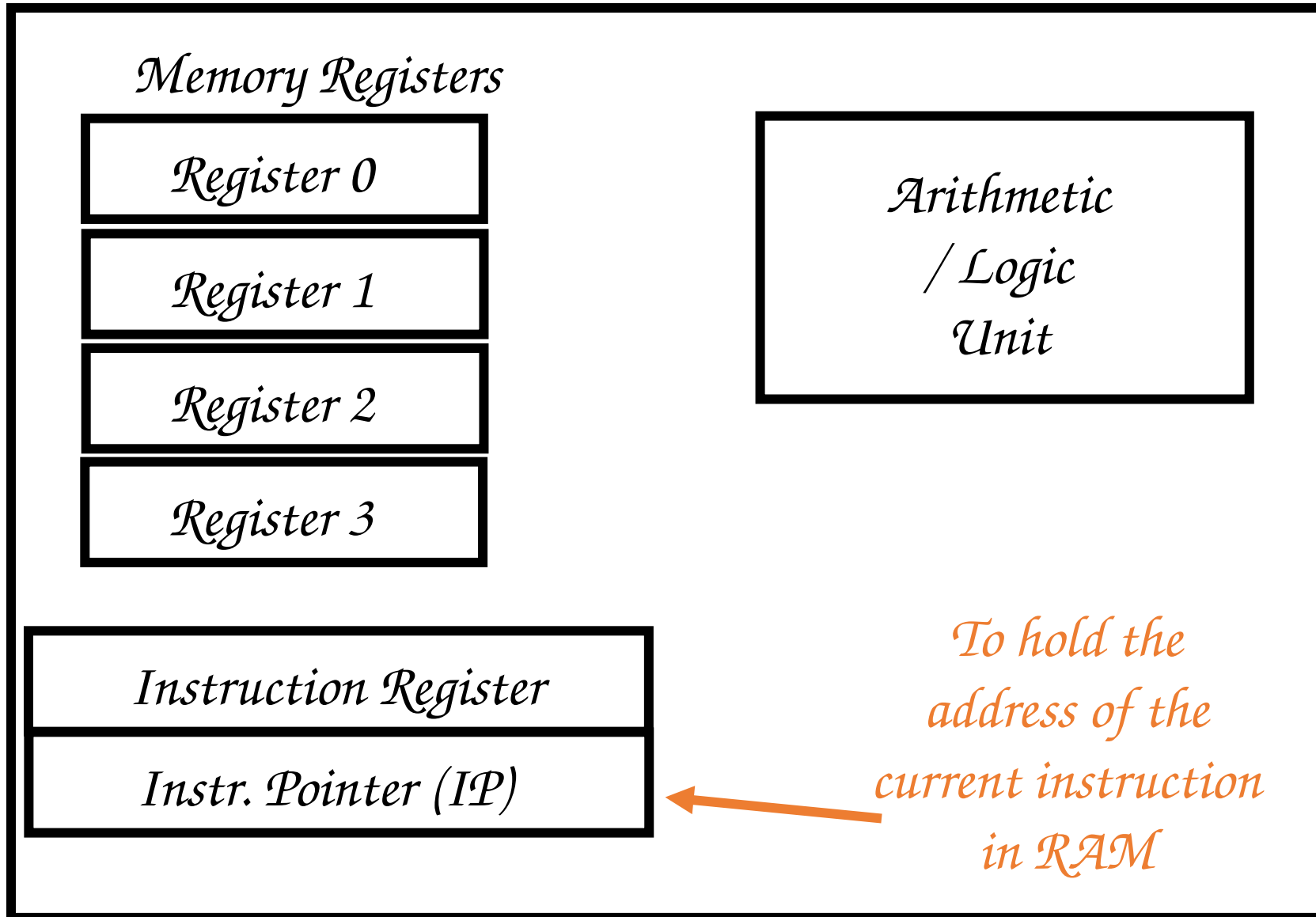
Register 3

*Arithmetic
/ Logic
Unit*



*For doing basic
Arithmetic / Logic
Operations on Values stored
in the Registers*





Memory Registers

Register 0

Register 1

Register 2

Register 3

Instruction Register

Instr. Pointer (IP)

*Arithmetic
/ Logic
Unit*

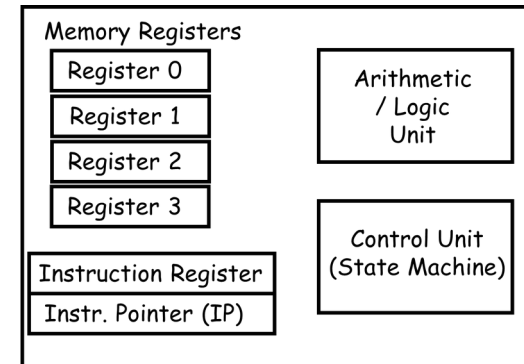
*Control Unit
(State Machine)*

High level code to machine instructions

```
1- BinarySearch(a, x)
2   left = 0
3   right = a.size()
4-  while left < right
5       middle = (left + right) / 2
6-      if a[middle] <= x then
7          left = middle + 1
8-      else
9          right = middle
10
11- if left > 0 and a[left - 1] == x then
12     return true
13- else
14     return false
```



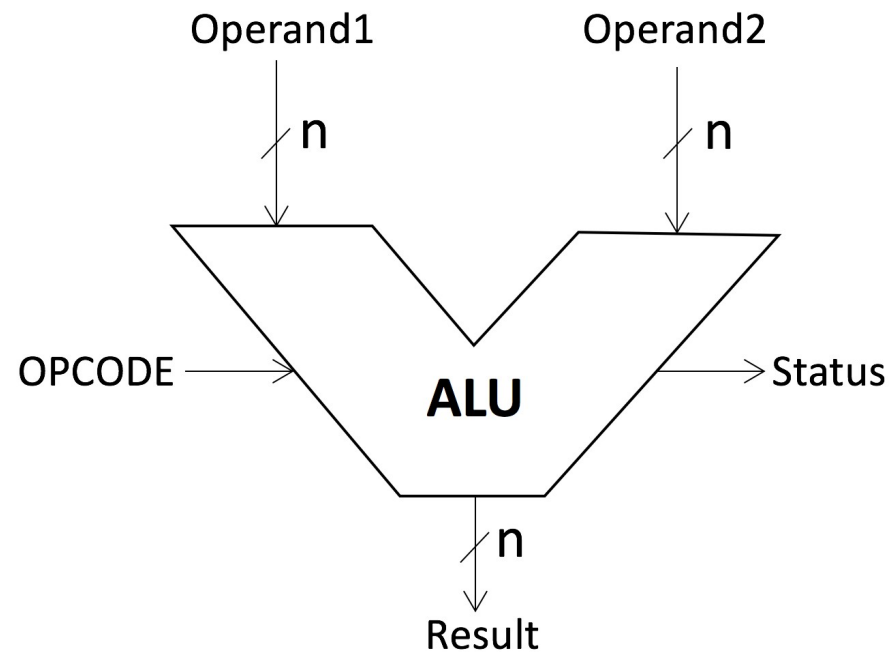
```
00000000 0000 0001 0001 1010 0010 0001 0004 0128
00000010 0000 0016 0000 0028 0000 0010 0000 0020
00000020 0000 0001 0004 0000 0000 0000 0000 0000
00000030 0000 0000 0000 0010 0000 0000 0000 0204
00000040 0004 8384 0084 c7c8 00c8 4748 0048 e8e9
00000050 00e9 6a69 0069 a8a9 00a9 2828 0028 fdfc
00000060 00fc 1819 0019 9898 0098 d9d8 00d8 5857
00000070 0057 7b7a 007a bab9 00b9 3a3c 003c 8888
00000080 8888 8888 8888 8888 288e be88 8888 8888
00000090 3b83 5788 8888 8888 7667 778e 8828 8888
000000a0 d61f 7abd 8818 8888 467c 585f 8814 8188
000000b0 8b06 e8f7 88aa 8388 8b3b 88f3 88bd e988
000000c0 8a18 880c e841 c988 b328 6871 688e 958b
000000d0 a948 5862 5884 7e81 3788 1ab4 5a84 3eec
000000e0 3d86 dcb8 5cbb 8888 8888 8888 8888 8888
000000f0 8888 8888 8888 8888 8888 8888 8888 0000
00001000 0000 0000 0000 0000 0000 0000 0000 0000
*
00001030 0000 0000 0000 0000 0000 0000 0000 0000
0000103e
```



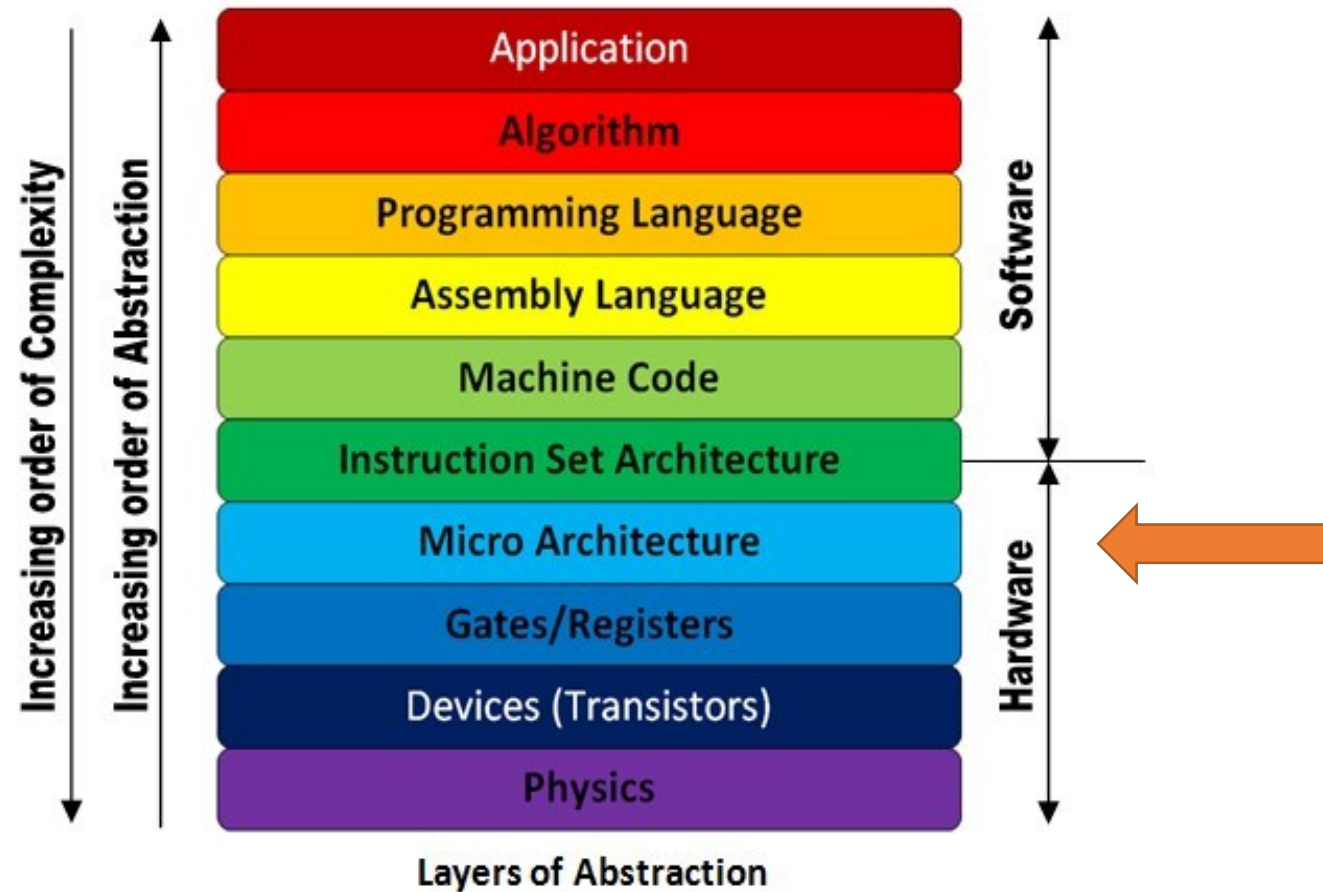
So far

- Combinational circuit:
 - Adder circuit
- Sequential circuit:
 - Register, shift-register

add operand1, opernad2
mov operand1, opernad2

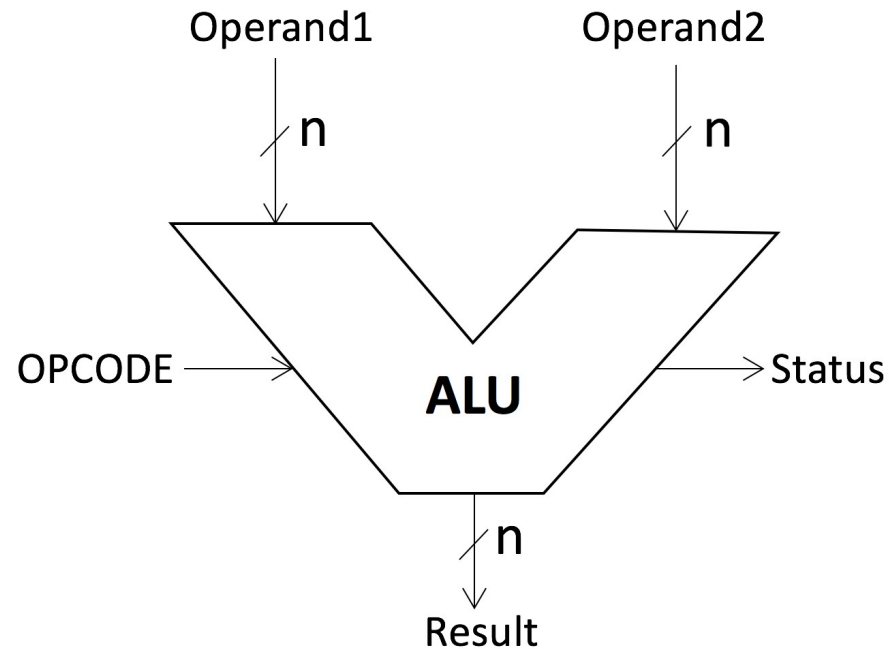


This lecture



Software-hardware interaction

Designing a computing circuit is like programming it.

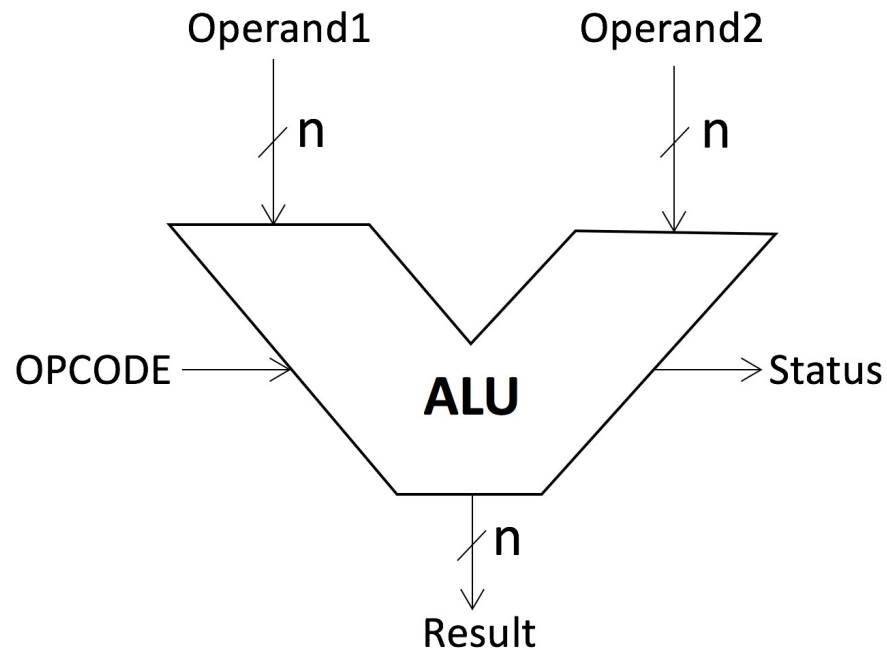


```
if(Opcode == ADD)
    perform add operation between
    Register R1 and Register R2 and
    store the result in Register R3
else if(Opcode == SUB)
    perform sub operation between
    Register R1 and Register R2 and
    store the result in Register R3
```

RTL or Register transfer language

- To express the hardware operations, computer scientist developed a new abstraction called RTL or Register Transfer Language
 - $R1 \leftarrow R2$
 - $R3 \leftarrow R1 + R2$

Design an arithmetic unit



$$R3 \leftarrow R1 + R2$$

Addition

$$R3 \leftarrow R1 - R2$$

Subtraction

$$R2 \leftarrow \overline{R2}$$

1's Complement

$$R2 \leftarrow \overline{R2} + 1$$

2's Complement

$$R3 \leftarrow R1 + \overline{R2} + 1$$

Subtraction!

$$R1 \leftarrow R1 + 1$$

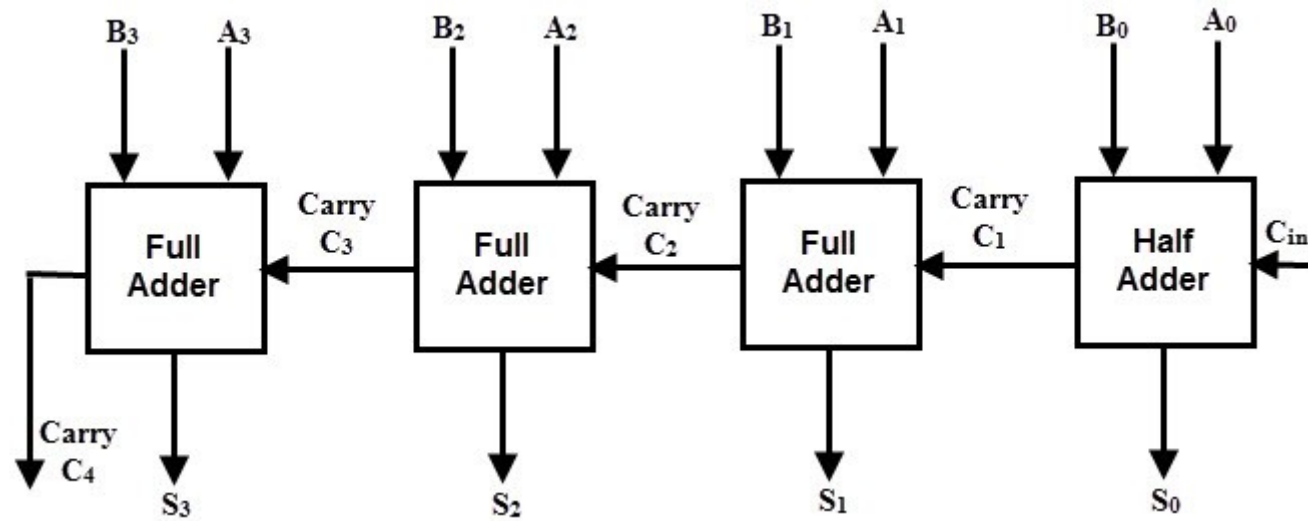
Increment

$$R1 \leftarrow R1 - 1$$

Decrement

Arithmetic microoperations

4bit Binary adder



4bit adder-subtractor

Input		Output
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

When B=1
output
complements A

When B=0
output
transmits A

$$R3 \leftarrow R1 + R2$$

Addition

$$R3 \leftarrow R1 - R2$$

Subtraction

$$R2 \leftarrow \overline{R2}$$

1's Complement

$$R2 \leftarrow \overline{R2} + 1$$

2's Complement

$$R3 \leftarrow R1 + \overline{R2} + 1$$

Subtraction!

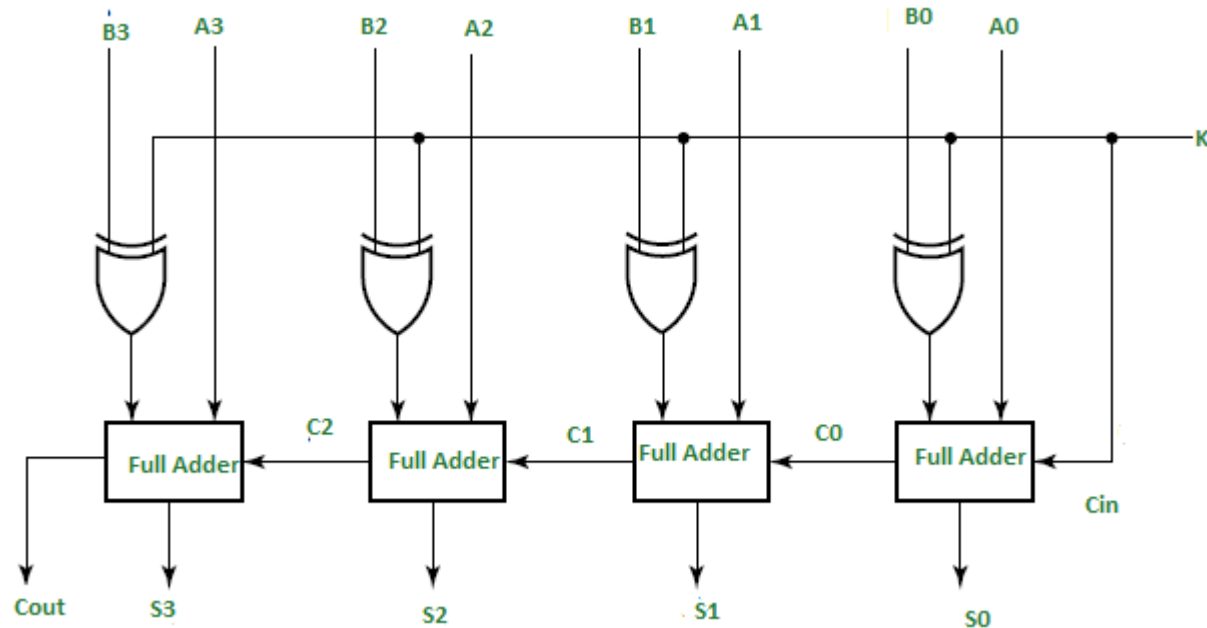
$$R1 \leftarrow R1 + 1$$

Increment

$$R1 \leftarrow R1 - 1$$

Decrement

4bit adder-subtractor



$$R3 \leftarrow R1 + R2$$

Addition

$$R3 \leftarrow R1 - R2$$

Subtraction

$$R2 \leftarrow \overline{R2}$$

1's Complement

$$R2 \leftarrow \overline{R2} + 1$$

2's Complement

$$R3 \leftarrow R1 + \overline{R2} + 1$$

Subtraction!

$$R1 \leftarrow R1 + 1$$

Increment

$$R1 \leftarrow R1 - 1$$

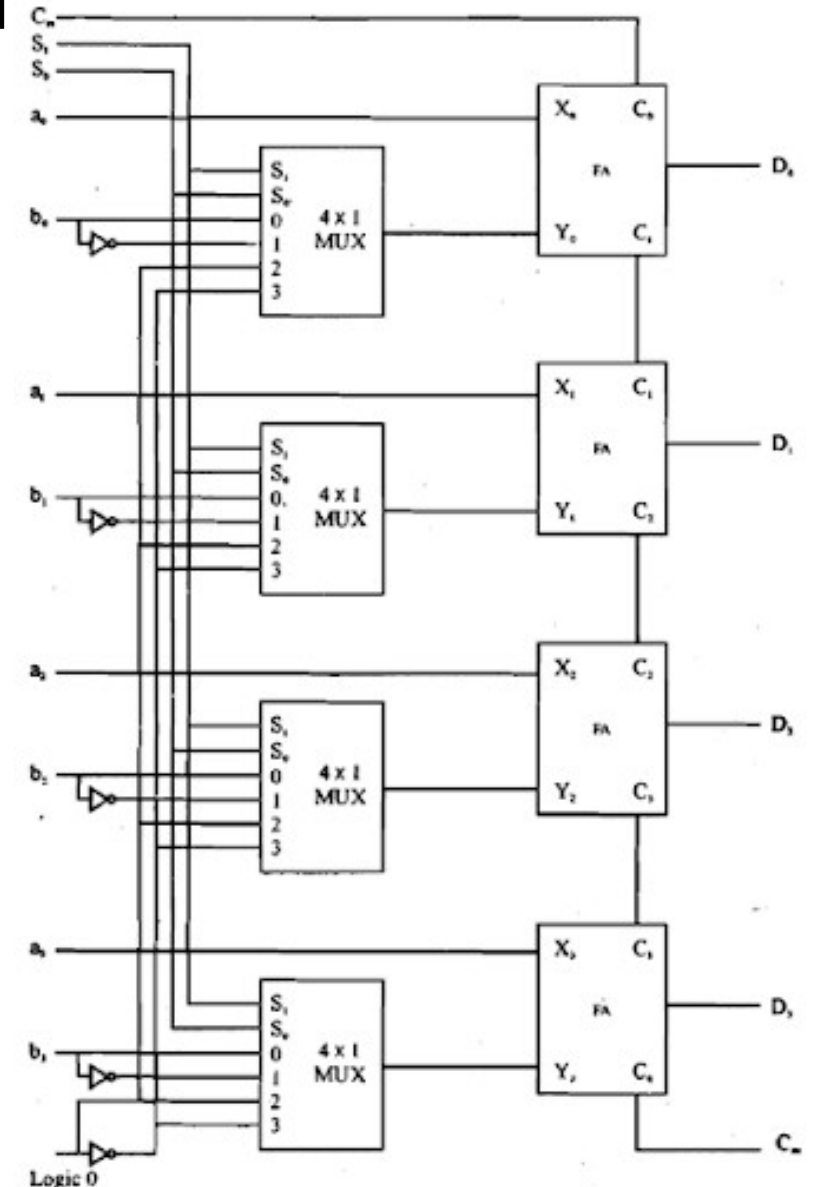
Decrement

Arithmetic circuit function table

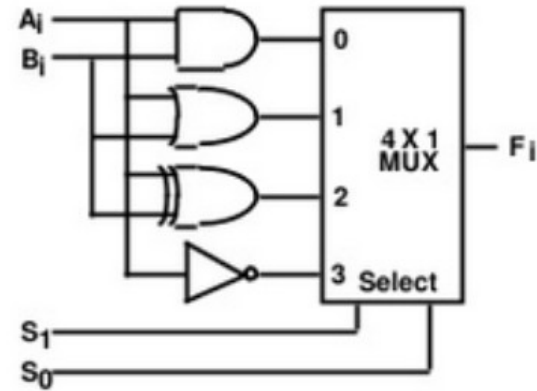
Select			Input	Output	Micro operation
S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with Carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with Borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Arithmetic circuit function table

Select			Input	Output	Micro operation
S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with Carry
0	1	0	$\overline{B}$	$D = A + \overline{B}$	Subtract with Borrow
0	1	1	$\overline{B}$	$D = A + \overline{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A



Logic units



Function table

S_1	S_0	Output	μ -operation
0	0	$F = A \wedge B$	AND
0	1	$F = A \vee B$	OR
1	0	$F = A \oplus B$	XOR
1	1	$F = A'$	Complement

So far...

- Arithmetic microoperations
- Logic microoperations
- How do they interact with other components in processor?
- How processor executes an instruction?

Memory Registers

Register 0

Register 1

Register 2

Register 3

Instruction Register

Instr. Pointer (IP)

*Arithmetic
/ Logic
Unit*

*Control Unit
(State Machine)*

The Control Unit

- It all comes down to the Control Unit.
- This is just a State Machine.
- How does it work?

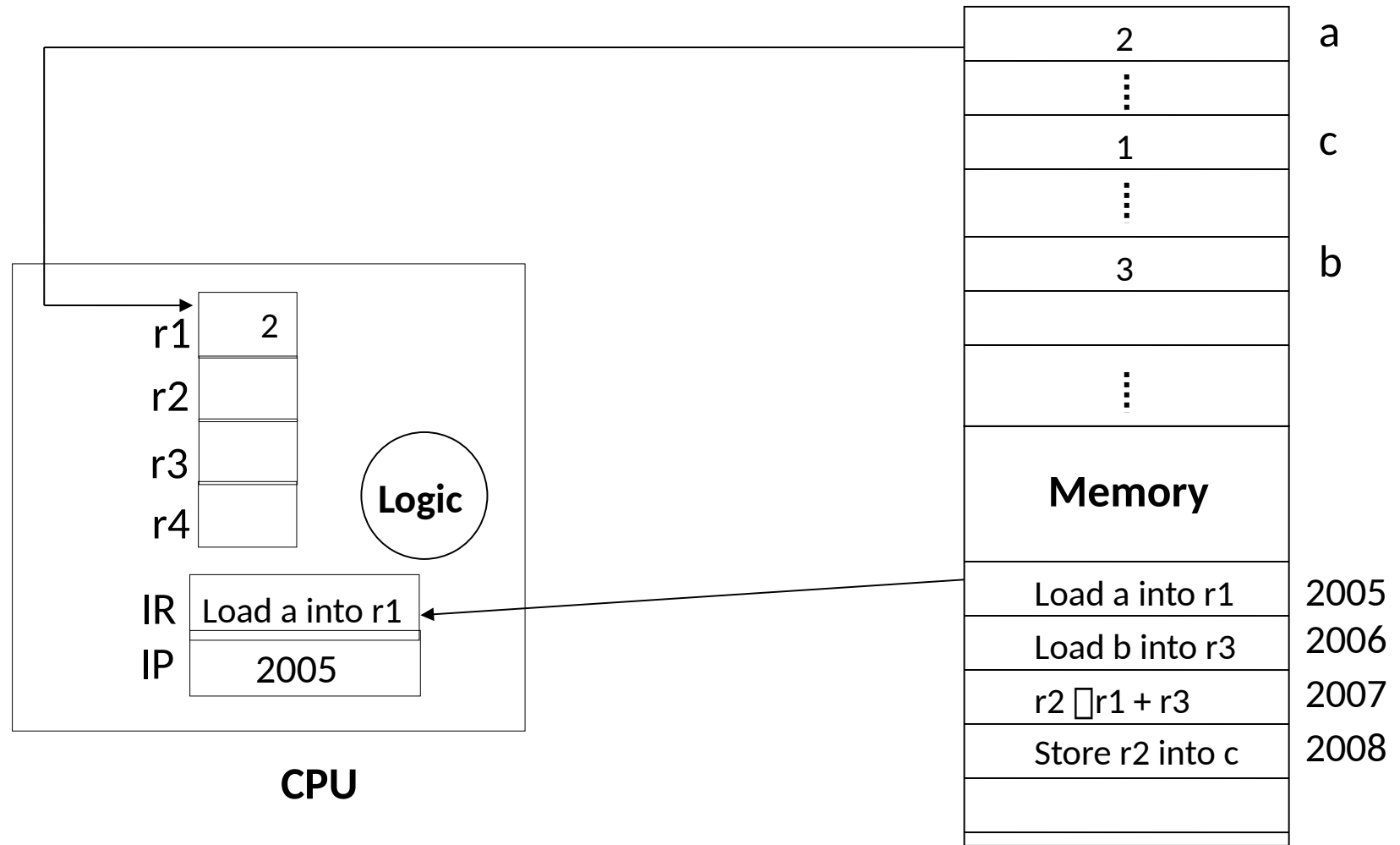
The control unit state machine has 5 states:

- Also known as instruction execution steps or instruction cycle
- 5 generic steps
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID/RF)
 - Execute/Evaluate memory address (EX/AG)
 - Memory operand fetch (MEM)
 - Store/writeback result (WB)

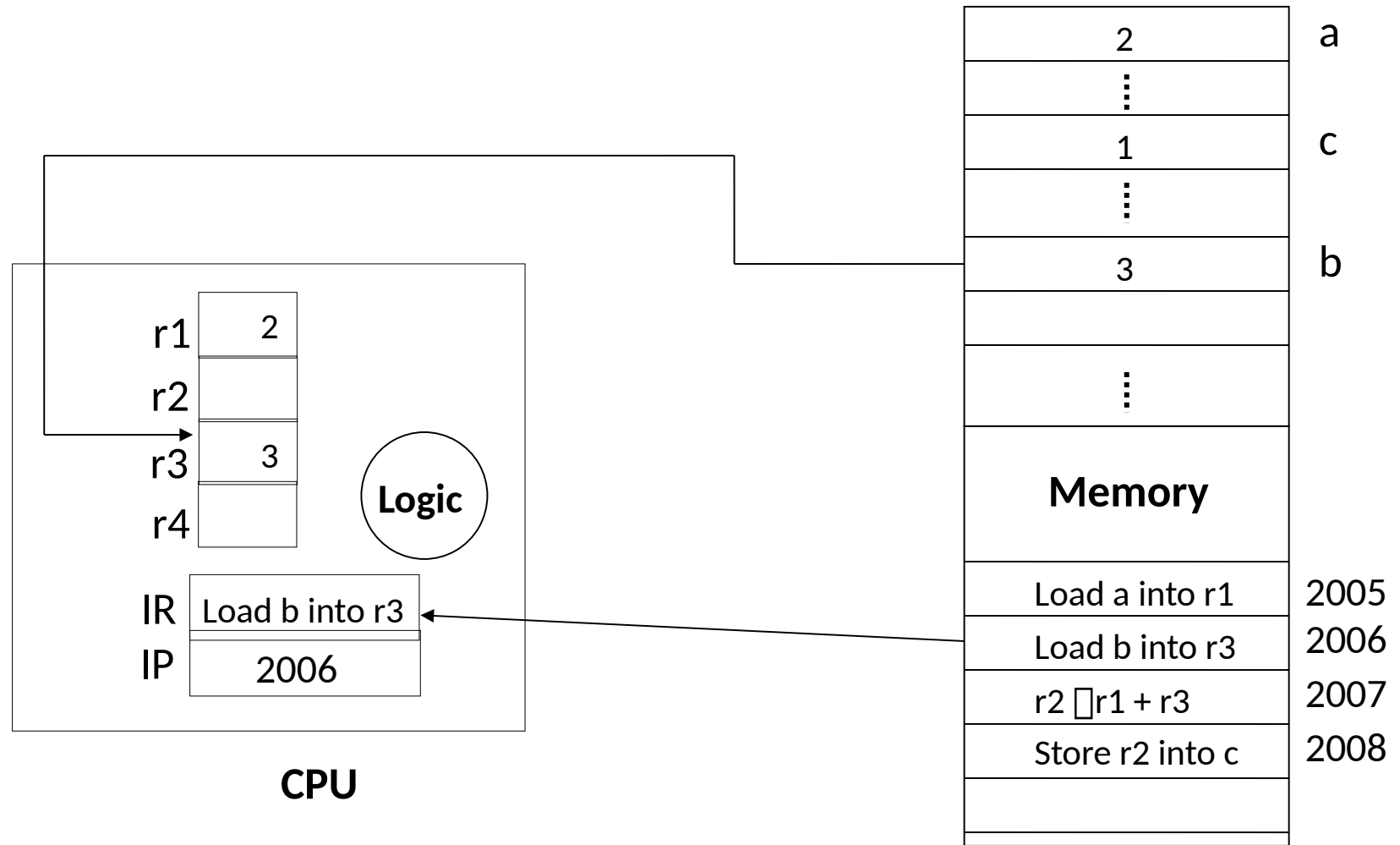
A Simple Program

- Want to add values of variables a and b (assumed to be in memory), and put the result in variable c in memory, i.e. $c \leftarrow a+b$
- Instructions in program
 - Load a into register r1
 - Load b into register r3
 - $r2 \leftarrow r1 + r3$
 - Store r2 in c

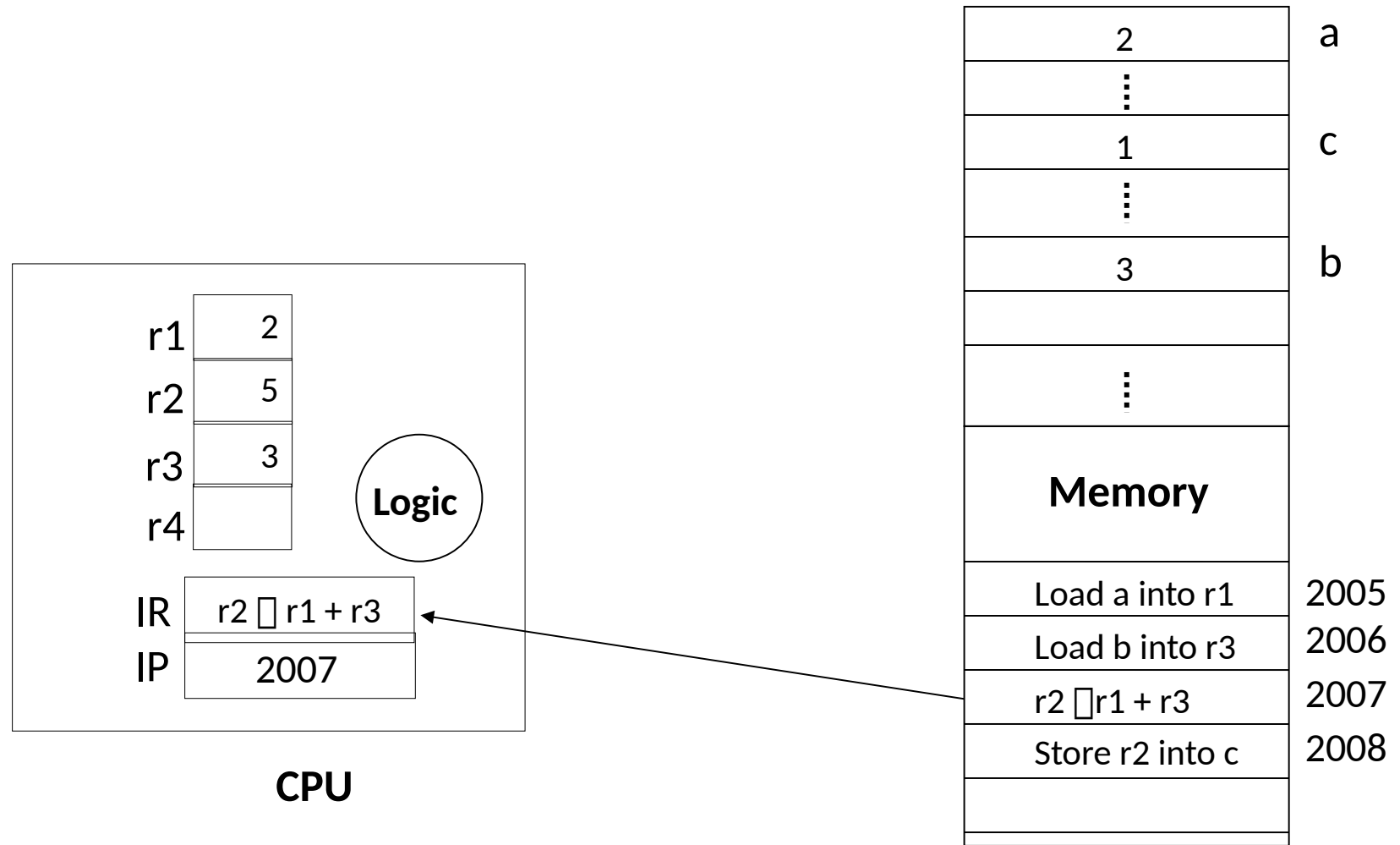
Running the Program



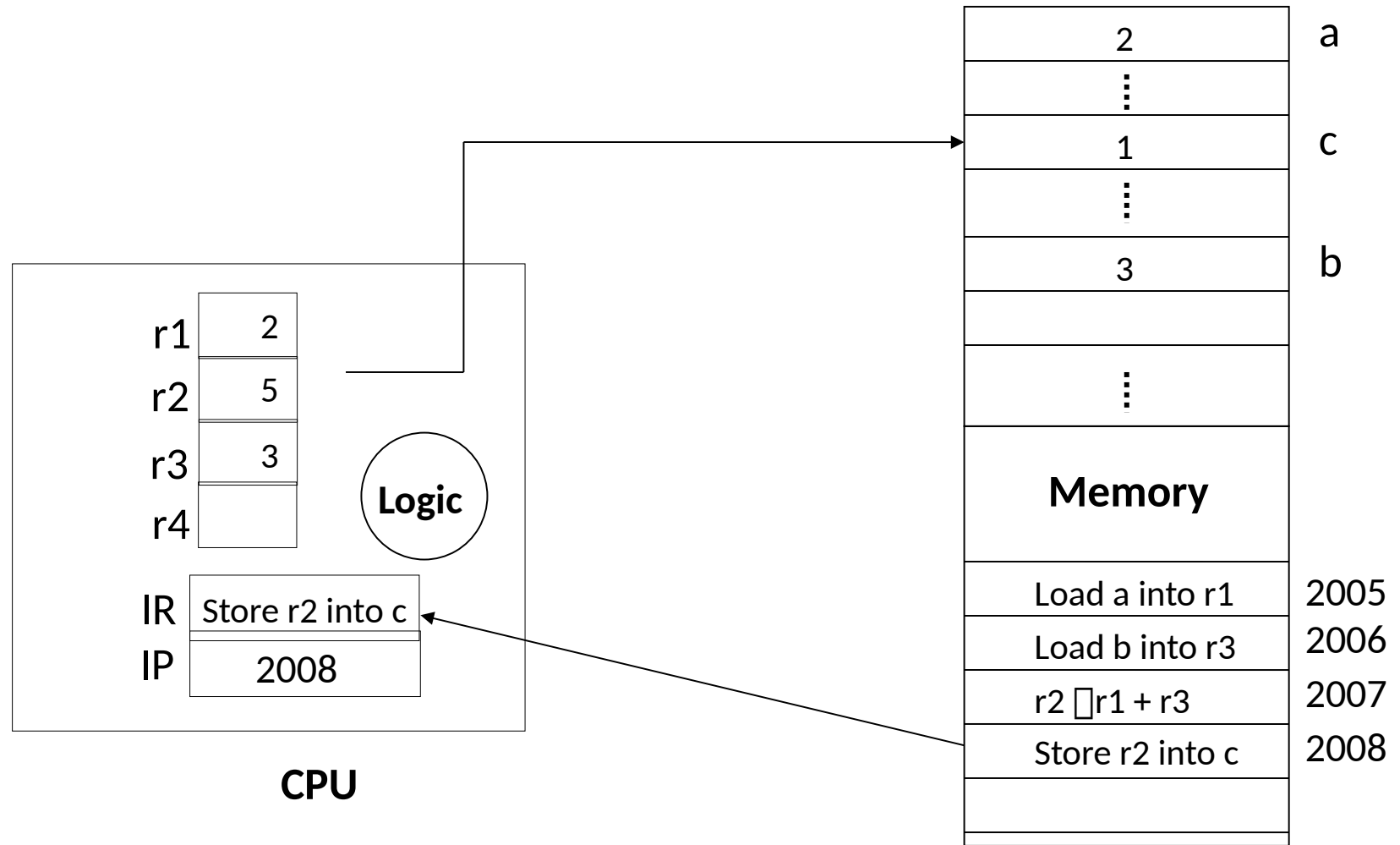
Running the Program



Running the Program



Running the Program



Running the Program

